CompleteGuide to Deploying Laravel Projects

Deploy your Laravel app like a pro without pulling your hair out.

📍 Table of Contents

1. Introduction
2. Requirements Checklist
3. Uploading Your Laravel Project
4. Configuring File Permissions
5. Setting Up the `.env` File
6. Installing Composer Dependencies
7. Building Frontend Assets (Optional)
8. Configuring Nginx or Apache
9. Enabling HTTPS with Let's Encrypt
10. Running Laravel Migrations & Seeds
11. Setting Up Cron Jobs
12. Managing Laravel Queues
13. Final Security & Performance Tips

---

1. 🚀 Introduction

Deploying a Laravel project doesn’t have to be complicated. Whether you're launching a new web app, an admin dashboard, or a SaaS platform, this guide gives you the exact steps to take your Laravel source code and put it online.

No fluff. No guesswork. Just results.

---

2. ✅ Requirements Checklist

Make sure your server or hosting environment includes:

Tool	Required Version
PHP	8.1 or higher
Composer	Latest version
MySQL	5.7+ / MariaDB
Nginx / Apache	Up to date
Node.js + npm	Optional (for asset build)

🔧 Example (Ubuntu/Debian):

bash
CopyEdit

```
sudo apt update
sudo apt install php php-cli php-mbstring php-xml php-bcmath php-curl
php-mysql unzip git curl composer nginx mysql-server -y
```

---

3. 📂 Uploading Your Laravel Project

You’ve got the source code — now get it on your server.

🔄 Option A: Upload via SCP

bash
CopyEdit

```
scp -r /local/path/laravel user@server-ip:/var/www/project-name
```

🔄 Option B: Clone from Git

bash
CopyEdit

```
cd /var/www
git clone https://github.com/your-repo/project-name.git
```

🔄 Option C: Use FTP (shared hosting)

- Upload your files to the root directory (`public_html` or `www`)
- Ensure `public/` content goes in root; rest above it.

---

4. 🔐 Configuring File Permissions

Laravel needs to write to specific folders. Set the right permissions:

bash
CopyEdit

```
cd /var/www/project-name
sudo chown -R www-data:www-data .
sudo chmod -R 775 storage bootstrap/cache
```

---

5. ⚙️ Setting Up the `.env` File

If it's not there, create it:

bash
CopyEdit

```
cp .env.example .env
```

Edit values:

env
CopyEdit

```
APP_NAME=MyLaravelApp
```

```
APP_ENV=production
APP_KEY=
APP_URL=https://yourdomain.com

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=mydb
DB_USERNAME=myuser
DB_PASSWORD=secret
```

---

6. 📦 Installing Composer Dependencies

From project root:

bash
CopyEdit

```
composer install --no-dev --optimize-autoloader
php artisan key:generate
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

---

7. 🧱 Building Frontend Assets (Optional)

If your project includes Vue, React, or Tailwind:

bash
CopyEdit

```
npm install
npm run build
```

If not using JS frameworks, skip this step.

---

8. 🌐 Configuring Nginx or Apache

🔧 Nginx Example Config

Create file:

bash
CopyEdit

```
sudo nano /etc/nginx/sites-available/project-name
```

Paste:

nginx
CopyEdit

```
server {
    listen 80;
    server_name yourdomain.com;
    root /var/www/project-name/public;

    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.x-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Enable it:

bash
CopyEdit

```
sudo ln -s /etc/nginx/sites-available/project-name
/etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

---

9. 🔒 Enabling HTTPS with Let's Encrypt

Let's keep things secure.

🔧 Certbot (Nginx Example):

bash
CopyEdit

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d yourdomain.com
```

SSL will auto-renew.

---

10. 🧬 Running Migrations & Seeds

Run database schema + optional seeders:

bash
CopyEdit

```
php artisan migrate --force
php artisan db:seed --force
```

---

11. ⏱️ Setting Up Cron Jobs (Laravel Scheduler)

Open crontab:

bash
CopyEdit

```
crontab -e
```

Add this line:

bash
CopyEdit
```
* * * * * cd /var/www/project-name && php artisan schedule:run >>
/dev/null 2>&1
```

This runs Laravel's `schedule:run` every minute.

---

12. ⚙️ Managing Laravel Queues (if used)

For background jobs:

bash
CopyEdit
```
php artisan queue:work
```

For Production: use Supervisor

Create file:

bash
CopyEdit
```
sudo nano /etc/supervisor/conf.d/laravel-worker.conf
```

Paste:

ini
CopyEdit
```
[program:laravel-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/project-name/artisan queue:work --sleep=3
--tries=3
autostart=true
autorestart=true
numprocs=1
user=www-data
redirect_stderr=true
```

```
stdout_logfile=/var/www/project-name/storage/logs/worker.log
```

Then:

bash
CopyEdit

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start laravel-worker:*
```

---

13. 🛡️ Final Security & Performance Tips

- 🔐 Never expose `.env` to the public.
- 🔄 Run `php artisan config:cache` after changes to `.env`
- ⚡ Use `php artisan optimize` to boost performance
- ✅ Use `fail2ban` + `ufw` or firewall rules to secure your VPS
- 📦 Consider setting up Redis for better queue and cache performance

---

🎉 You're Live!

Your Laravel project should now be fully deployed and accessible on your domain. If something breaks, 90% of the time it's:

- Incorrect file permissions
- Wrong document root (should point to `public/`)
- Missing `.env` config
- Uncached or outdated configs